

量化交易Agent系列

AI驱动的智能交易系统

AI技术专栏

2026年

目录

1. 量化交易Agent架构设计

1. 量化交易Agent架构设计

量化交易Agent架构设计

系列导航：

【本地大模型知识库系列】 01-12篇（已完成） |

【英语学习Agent系列】 01-08篇（已完成） |

【量化交易Agent系列】 当前第01篇（共12篇）

量化交易Agent架构设计

AI驱动的智能交易系统

量化交易Agent系列第01篇 | 架构设计

在前面的【本地大模型知识库系列】和【英语学习Agent系列】中，我们学习了如何搭建本地大模型环境和构建垂直领域的智能Agent。现在，让我们将这些技术应用到金融领域——打造一个量化交易Agent。

01

传统量化交易的痛点

传统的量化交易方式存在很多问题：

传统量化的痛点

-

因子挖掘困难：

传统方法依赖分析师经验，难以发现非线性关系和隐藏模式

-

策略过拟合：

历史数据优化后实盘表现差，缺乏鲁棒性

-

信息处理局限：

难以有效处理新闻、研报等非结构化文本数据

-

决策延：

市场变化快，人工调整策略响应不及时

-

风险控制被动：

缺乏实时风险预警和智能风控机制

02

AI量化交易的优势

一个智能的量化交易Agent可以解决这些问题：

AI量化交易的优势

-

智能因子挖掘：

利用大模型发现隐藏模式和非线性关系

-

自适应策略优化：

基于实时数据动态调整策略参数

-

多模态信息融合：

整合新闻、研报、社交情绪等非结构化数据

-

实时风险预警：

AI驱动的风险识别和预警机制

-

自动化执行：

从信号生成到下单执行全流程自动化

03

量化交易Agent架构

我们的量化交易Agent基于之前搭建的本地知识库系统，架构如下：

整个系统分为六层：

数据采集层：

获取股票行情、财务数据、新闻资讯等

数据处理层：

数据清洗、因子计算、特征工程

策略引擎层：

多因子策略、趋势跟踪、均值回归等策略实现

大模型增强层：

使用Ollama运行本地大模型进行情绪分析和决策辅助（参考【本地大模型知识库系列】第02篇）

风控管理层：

仓位限制、止损止盈、风险度量

执行监控层：

实盘下单、持仓管理、绩效监控（使用Gradio构建界面，参考第04/09篇）

04

核心功能模块

系统包含八个核心功能模块：

模块

功能描述

技术实现

数据获取

行情数据、财务数据、新闻资讯

Tushare、AKShare、爬虫

因子计算

量价因子、财务因子、另类因子

Pandas、NumPy向量化计算

策略引擎

多因子策略、趋势跟踪、均值回归

策略模板+信号生成

回测系统

历史数据回测、绩效评估、参数优化

事件驱动回测框架

风控管理

仓位限制、止损止盈、VaR计算

风控规则引擎

AI增强

新闻情绪分析、因子挖掘、风险预警

Ollama + LLM（参考第02篇）

实盘接口

券商API对接、下单执行、成交查询

REST API、WebSocket

监控界面

实时行情、持仓状态、绩效仪表盘

Gradio（参考第04/09篇）

05

核心代码结构

项目的核心代码结构如下：

python

class

QuantTradingAgent

:

```
def
__init__
(
self
):
self
.
data_provider

None

self
.
factor_engine

None

self
.
strategy_engine

None

self
.
backtester

None

self
.
risk_manager
```

None

self

.

ai_enhancer

None

self

.

execution_engine

None

self

.

_init_components

()

def

_init_components

(

self

):

数据获取模块

self

.

data_provider

DataProvider

()

因子计算模块

self

.

factor_engine

FactorEngine

()

策略引擎模块

self

.

strategy_engine

StrategyEngine

()

回测模块

self

.

backtester

Backtester

()

风控模块

self

.

risk_manager

RiskManager

()

AI增强模块（基于Ollama）

self

.

ai_enhancer

AIEnhancer

()

执行模块

self

.

execution_engine

ExecutionEngine

()

def

run_backtest

(

self

,

strategy_name

,

params

):

获取历史数据

data

self

.

data_provider

.

get_history_data

()

计算因子

factors

self

.

factor_engine

.

calculate_factors

(

data

)

生成信号

signals

self

.

strategy_engine

.

generate_signals

(

factors

,

params

)

```
# 执行回测
```

```
results
```

```
self
```

```
.
```

```
backtester
```

```
.
```

```
run
```

```
(
```

```
signals
```

```
,
```

```
data
```

```
)
```

```
return
```

```
results
```

```
def
```

```
run_live
```

```
(
```

```
self
```

```
,
```

```
strategy_name
```

```
,
```

```
params
```

```
):
```

```
# 实时数据获取
```

```
realtime_data
```

```
self
.  
data_provider  
.  
get_realtime_data  
(
```

```
# AI情绪分析  
sentiment
```

```
self  
.  
ai_enhancer  
.  
analyze_sentiment  
(  
realtime_data  
)
```

```
# 生成交易信号  
signal
```

```
self  
.  
strategy_engine  
.  
generate_signal  
(  
realtime_data
```

```
,  
  
sentiment  
  
)  
  
# 风控检查  
  
if  
  
not  
  
self  
  
.  
  
risk_manager  
  
.  
  
check_risk  
  
(  
  
signal  
  
):  
  
return  
  
"风控拦截"  
  
# 执行交易  
  
result  
  
  
self  
  
.  
  
execution_engine  
  
.  
  
execute  
  
(  
  
signal  
  
)
```

return

result

常见问题

1. 需要什么样的硬件配置？：建议至少16GB内存，有GPU更好（参考【本地大模型知识库系列】第02篇）
2. 数据安全吗？：所有数据存储在本地，无需上传到云端（参考【本地大模型知识库系列】第01篇）
3. 需要券商API吗？：回测不需要，实盘交易需要申请券商API权限
4. 支持哪些市场？：本系列以A股为主，可扩展到美股、港股等

—— 未完待续 ——

系列导航：

- 本地大模型知识库系列（12篇，已完成）
- 英语学习Agent系列（8篇，已完成）
- 量化交易Agent系列（12篇，当前第01篇）

量化交易Agent系列第01篇

下一篇：数据获取与预处理——搭建量化数据基础设施